

SQL Aggregations & Analytical Functions

5 · Aggregations & Analytical Functions — Complete Reference Guide (Banking Project)

Table of Contents

1. Aggregation Functions
2. Window Function Basics
3. Window Aggregation Functions
4. Window Ranking Functions
5. Window Value Functions

1. Aggregation Functions

→ **Task 1: Generate a report showing the total sales amount, average transaction amount, and transaction count for each transaction type (Aggregate Data).**

```
SELECT
    transaction_type,
    SUM(transaction_amount) AS total_amount,
    AVG(transaction_amount) AS avg_amount,
    COUNT(*) AS txn_count
FROM Transactions
GROUP BY transaction_type;
```

→ **Task 2: Find the minimum and maximum account balance for each customer segment (Aggregate & Filter Groups).**
Only include segments with more than 5 accounts.

```
SELECT
    c.customer_segment,
    MIN(a.account_balance) AS min_balance,
    MAX(a.account_balance) AS max_balance,
    COUNT(*) AS account_count
FROM Customers c
INNER JOIN Accounts a
    ON c.customer_id = a.customer_id
GROUP BY c.customer_segment
HAVING COUNT(*) > 5;
```

2. Window Function Basics

→ **Task 3: Find the total number of transactions overall and the total number of transactions for each account (Window Function Basics).**

```
SELECT
    transaction_id, account_id, transaction_date,
    COUNT(*) OVER () AS total_transactions,
    COUNT(*) OVER (PARTITION BY account_id) AS transactions_by_account
FROM Transactions;
```

→ **Task 4: Check the Accounts table for duplicate Account IDs (Window Function Basics).**

```
SELECT *
FROM (
    SELECT *,
        COUNT(*) OVER (PARTITION BY account_id) AS duplicate_check
    FROM Accounts
) t
WHERE duplicate_check > 1;
```

3. Window Aggregation Functions

→ **Task 5: Find the total and average transaction amount overall and for each account (Window Aggregation Functions).**

```
SELECT
    transaction_id, account_id, transaction_amount,
    SUM(transaction_amount) OVER ()
        AS total_amount_overall,
    SUM(transaction_amount) OVER (PARTITION BY account_id)
        AS total_amount_by_account,
    AVG(transaction_amount) OVER (PARTITION BY account_id)
        AS avg_amount_by_account
FROM Transactions;
```

→ **Task 6: Find the lowest and highest transaction amount for each account (Window Aggregation Functions).**
Additionally, calculate how far each transaction is from both values.

```
SELECT
    transaction_id, account_id, transaction_amount,
    MIN(transaction_amount) OVER (PARTITION BY account_id)
        AS lowest_for_account,
    MAX(transaction_amount) OVER (PARTITION BY account_id)
        AS highest_for_account,
    transaction_amount
        - MIN(transaction_amount) OVER (PARTITION BY account_id)
        AS diff_from_lowest,
    MAX(transaction_amount) OVER (PARTITION BY account_id)
        - transaction_amount AS diff_from_highest
FROM Transactions;
```

4. Window Ranking Functions

→ **Task 7: Rank each account based on their balance within their customer segment, from highest to lowest (Window Ranking Functions).**

Additionally, provide details such as Account ID and Customer Segment.

```
SELECT
  c.customer_segment, a.account_id, a.account_balance,
  ROW_NUMBER() OVER (
    PARTITION BY c.customer_segment
    ORDER BY a.account_balance DESC
  ) AS row_num,
  RANK() OVER (
    PARTITION BY c.customer_segment
    ORDER BY a.account_balance DESC
  ) AS rank_num
FROM Customers c
INNER JOIN Accounts a
  ON c.customer_id = a.customer_id;
```

→ **Task 8: Rank each account within their segment without leaving gaps when balances tie, and divide each segment's accounts into 4 equal groups based on balance (Window Ranking Functions).**

```
SELECT
  c.customer_segment, a.account_id, a.account_balance,
  DENSE_RANK() OVER (
    PARTITION BY c.customer_segment
    ORDER BY a.account_balance DESC
  ) AS dense_rank_num,
  NTILE(4) OVER (
    PARTITION BY c.customer_segment
    ORDER BY a.account_balance DESC
  ) AS balance_quartile
FROM Customers c
INNER JOIN Accounts a
  ON c.customer_id = a.customer_id;
```

5. Window Value Functions

→ **Task 9: Find the previous and next transaction amount for each account, ordered by transaction date (Window Value Functions).**

Additionally, provide details such as Account ID and Transaction Date.

```
SELECT
    account_id, transaction_id, transaction_date, transaction_amount,
    LAG(transaction_amount) OVER (
        PARTITION BY account_id ORDER BY transaction_date
    ) AS previous_amount,
    LEAD(transaction_amount) OVER (
        PARTITION BY account_id ORDER BY transaction_date
    ) AS next_amount
FROM Transactions;
```

→ **Task 10: Find the first and last transaction amount for each account (Window Value Functions).**

```
SELECT
    account_id, transaction_id, transaction_date, transaction_amount,
    FIRST_VALUE(transaction_amount) OVER (
        PARTITION BY account_id ORDER BY transaction_date
    ) AS first_transaction_amount,
    LAST_VALUE(transaction_amount) OVER (
        PARTITION BY account_id ORDER BY transaction_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS last_transaction_amount
FROM Transactions;
```